ANKARA YILDIRIM BEYAZIT UNIVERSITY

CENG303 - DESIGN AND ANALYSIS OF ALGORITHMS

BONUS HOMEWORK - REPORT

Fatih Mehmet UNAL - 22050151009
Elif EROGLU - 20050111080

Date : 12.12.2025

1. Boyer–Moore Algorithm Implementation

We implemented the classic Boyer–Moore algorithm in Java. While working on it, we used the explanations from GeeksforGeeks, where Boyer–Moore is presented in two versions: one that uses only the bad-character rule and another that uses the good-suffix rule. We combined both ideas to produce a full Boyer–Moore implementation that applies both heuristics. We also asked LLMs for general advice about the algorithms, but we did not use them to write any part of the code.

2. Pre-Analysis Strategy

For the pre-analysis strategy, we considered the strengths of each algorithm. For example, the naive algorithm works well on short texts, while KMP is better when the pattern contains repeated prefixes. However, since the test case texts were quite short, the pre-analysis ended up selecting the naive algorithm most of the time. If the dataset had included much longer texts, our decisions and comparisons would likely have been more meaningful. Similarly, we consulted LLMs for conceptual guidance, but we did not use them to generate any code.

3. Performance Observations

- **The Naive algorithm** performs unexpectedly fast on small and medium-sized inputs.
- **KMP** provides the best results when the pattern contains repetitive prefixes or overlapping matches.
- **Rabin–Karp** performs particularly well in scenarios that are suitable for hashing.
- **Boyer–Moore** stands out in long-text cases, although its preprocessing cost is relatively high.
- Since **Pre-Analysis** relies on heuristic rules, it sometimes makes accurate choices and sometimes selects less optimal algorithms.

4. Experimental Result

Algorithm	**Result**
Naive	PASS (30/30)
KMP	PASS (30/30)
Rabin–Karp	PASS (30/30)
Boyer–Moore (our implementation)	PASS (29/30)
GoCrazy	Not implemented

- **Average Runtime Comparison**

  KMP: 5.828 µs

  Naive : 6.017 µs

  Rabin–Karp: 5.784 µs

  Boyer–Moore : 7.849 µs

  GoCrazy : -

5. Research and Usage

- Lecture Materials - Notes and Videos from Aybuzem
- GeeksforGeeks for searching Algorithms and Competitive Programming

6. Our Journey - What We Learned

From this homework, we learned that when the text size is very small, the naive algorithm is often the most efficient option for string matching. We also realized that to meaningfully observe expected time-complexity differences, we need significantly larger datasets.